

Sistemas Operativos

Facultad de Informática
Universidad Complutense de Madrid

Módulo 2 - Sistemas de ficheros



Outline

1 Introducción

2 Ficheros

3 Directorios

4 API del SO

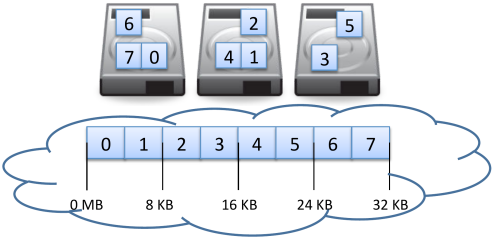
5 Sistema de gestión de ficheros

¿Qué es un fichero?

- Un fichero es una unidad de almacenamiento lógico no volátil que agrupa un conjunto de información relacionada entre sí bajo un mismo nombre
- Los ficheros suelen servir...
  - para almacenar código máquina o código fuente
  - como fuente de entrada a los programas
  - para guardar a largo plazo las salidas de los programas
- El sistema de Gestión de Ficheros se encarga de:
  - Ofrecer servicios de manipulación de ficheros a los programas
  - Gestionar los permisos de acceso
  - Garantizar la integridad de atributos y contenidos
  - Soportar los ficheros sobre medios de almacenamiento

Visión lógica vs. visión física

- Visión lógica:
  - Ficheros
  - Directorios
  - Sistemas de ficheros y particiones
- Visión física:
  - Bloques o bytes ubicados en dispositivos



Características para el usuario

- Almacenamiento permanente de información. Los ficheros no desaparecen aunque se apague el computador
  - Conjunto de información estructurada de forma lógica según criterios de aplicación
- Nombres lógicos y estructurados
- No están ligados al ciclo de vida de una aplicación particular
- Se accede a ellos a través de llamadas al sistema operativo o de bibliotecas de utilidades
- Abstraen los dispositivos de almacenamiento físico

Ficheros: visión del usuario

- Un espacio lógico de direcciones contiguas usado para almacenar datos

Clasificaciones de ficheros

- Por codificación de los datos almacenados:
  - Texto o Caracteres
  - Binarios
- Programas:
  - Código fuente
  - Ficheros objeto (imagen de carga)
- Documentos (específico de aplicación)



Atributos del fichero



- **Nombre**: la única información en formato legible por una persona
- **Identificadores**: ID del descriptor interno del fichero, ID propietario y grupo del fichero
- **Tipo de fichero (para el SO)**: necesario en sistemas que proporcionan distintos formatos de ficheros: normales y especiales
  - Para más información `man 2 stat`
- **Tamaño del fichero**: número de bytes en el fichero, máximo tamaño posible, etc.
- **Protección**: control de acceso (`rwx` para usuario, grupo y todos)
- **Información temporal**: de creación, de acceso, de modificación, etc.
- ...

Tipos de ficheros en UNIX

- Los SSOO tipo UNIX (Linux, Mac OS X, Solaris, ...) modelan como ficheros distintas abstracciones del SO
  - No todos los ficheros que pueden manipular los programas de usuario son ficheros "normales"

Tipos de ficheros en UNIX

- Regulares (normales)
- Directorios
- Dispositivos de caracteres
- Dispositivos de bloques
- Enlaces simbólicos
- Tuberías con nombre (FIFOs)
- ...

Descriptores de un fichero

- Cada objeto/entidad gestionada por el SO tiene un nombre o descriptor único
- Cada fichero posee dos descriptores únicos:
  - **Descriptor lógico**: nombre del fichero
  - **Descriptor físico**: estructura de datos interna que el SO manipula para realizar operaciones sobre el fichero
- Los directorios permiten al SO asociar el descriptor lógico de un fichero con su descriptor físico

Más sobre descriptores lógicos (nombres)

Los convenios de nombrado son específicos de cada SO

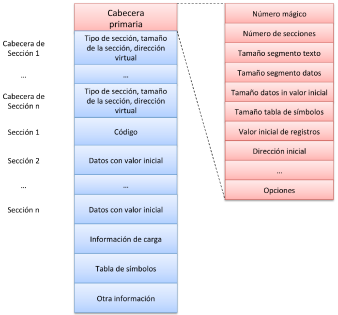
- **Longitud máxima**: 8 caracteres en `MS-DOS`, 255 en UNIX (4096 ruta completa)
- **Sensibles a tipografía**: `MiFichero` y `mifichero` son el mismo fichero en Windows pero distintos en LINUX
- **Extensiones**
  - Obligatoria o no
  - Más de una o no
  - Fija para cada tipo de documentos
  - En general, las extensiones sólo son significativas para las aplicaciones (pdf, doc, html, c, cpp, ...)

Más sobre descriptores físicos

- Cuando se crea un fichero, el SO crea una estructura de datos interna para el fichero (*descriptor físico*)
  - Se almacena en disco (*persistente*)
  - Se carga en memoria cuando el SO accede al fichero en nombre de algún programa de usuario
- Algunos sistemas de ficheros asocian IDs numéricos a descriptores físicos
  - Usado como un "puntero" al descriptor físico
  - Ejemplo: numero de nodo-i (UNIX)

Contenido de un fichero para el SO

- Desde el punto de vista del SO, la mayor parte de los ficheros son una colección de bytes desperdigada por uno o varios dispositivos de almacenamiento
- El SO reconoce solamente unos pocos formatos de fichero:
  - Ficheros ejecutables
  - Ficheros de texto
- (p.ej., scripts)



Sistemas de ficheros

El acceso a bajo nivel a los dispositivos es ...

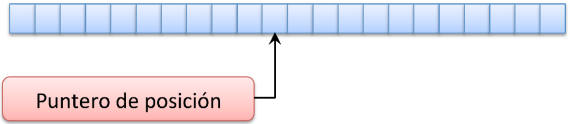
- Incómodo
  - Detalles físicos de los dispositivos
  - Dependiente de las direcciones físicas
- No seguro
  - Si el usuario accede al nivel físico no tiene restricciones

El sistema de gestión de ficheros

- Suministra una *visión lógica* de los dispositivos
- Ofrece primitivas de acceso cómodas e independientes de los detalles físicos (crear, borrar, leer, escribir, modificar, etc.)
- Implementa mecanismos de protección y recuperación frente a fallos
- Garantiza velocidad y eficiencia de uso

Ficheros: visión lógica y física

- Usuario: representación lógica (array de bytes)



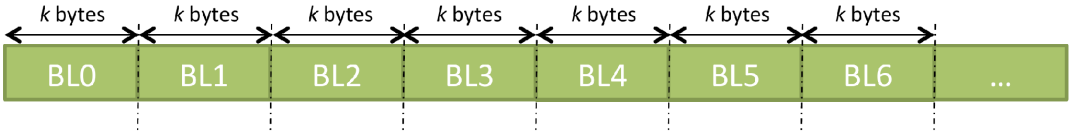
- SO: visión física ligada a dispositivos (conjunto de bloques)

Fichero A	
Bloques:	13
	20
	1
	8
	3
	16
	19

1	2	**3**	4	5	6
7	**8**	9	10	11	12
13	14	15	**16**	17	18
19	**20**	21	22	23	24
25	26	27	28	29	30

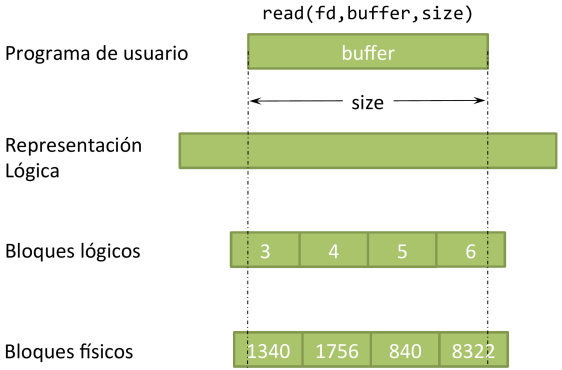
Bloque lógico vs. bloque físico

- El array de bytes de la representación lógica se divide en *bloques lógicos*
  - Si el tamaño de bloque es *k bytes*, un bloque lógico es un conjunto de *k bytes* contiguos en el fichero



- Un bloque físico representa una dirección de disco donde pueden almacenarse los datos de un bloque lógico

Operaciones sobre ficheros



Traducción dir. lógica → dir. física

- Los programas de usuario acceden a los datos de los ficheros empleando una dirección lógica (en bytes)
  - Para acceder a los datos, el SO debe obtener la dirección física correspondiente a la dirección lógica (traducción)
- Mecanismo de traducción
  - Dir. lógica → Dir. bloque lógico = (Núm. bloque lógico, Offset)
    - Num. bloque lógico = $\lfloor$ (DirLógica(bytes))/(TamBloque(bytes)) $\rfloor$
    - Offset = DirLógica(bytes) **mod** TamBloque(bytes)
  - Dir. bloque lógico → Dirección física = (Num. bloque físico, Offset)
    - Para establecer la correspondencia entre un bloque lógico y su bloque físico asociado se emplean estructuras de datos específicas de cada sistema de ficheros

Organización interna del sistema de ficheros







- ¿Cómo hacemos corresponder esos bloques con la imagen de fichero que tiene la aplicación?
- ¿Cómo asignamos los bloques de disco a un fichero cuando éste necesita más espacio?

Estructuras de datos (I)

- El SO necesita un conjunto de estructuras datos para poder manipular ficheros y sistemas de ficheros

Estructuras de datos en disco

- El sistema de ficheros ha de ser autocontenido, no volátil e independiente del SO
- Además de los datos es preciso almacenar metadatos en el disco
  - Estructuras para llevar la cuenta de bloques libres y ocupados
  - Estructuras para realizar la correspondencia entre direcciones lógicas (en bytes) y direcciones físicas (bloque físico, desplazamiento)
  - ...

Estructuras de datos (II)

Los datos han de estar memoria cuando sea necesario manipularlos

Estructuras de datos en memoria

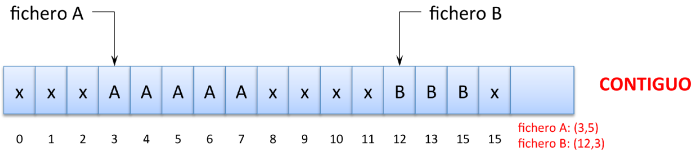
El SO mantiene estructuras de datos en memoria para poder realizar lecturas/escrituras sobre datos y metadatos más eficientemente * **No todos los datos de disco están en memoria**



Estrategias de organización de ficheros

- Organización contigua
  - Usada en CD-ROM y cintas
- Organización enlazada
  - Usada en sistemas FAT (File Allocation Table)
- Organización indexada
  - Típica de UNIX-SV, FFF (Fast File System), ext2, etc.
- Organizaciones basadas en árboles equilibrados
  - NTFS, JFS, Reiser, XFS, etc.

Organización contigua



Ventajas

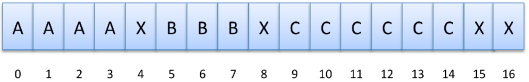
- Acceso secuencial óptimo
- Permite lecturas anticipadas
- Implementación sencilla de accesos aleatorios

Desventajas

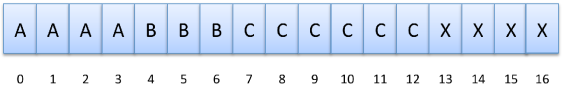
- Redimensionar un fichero puede ser costoso
- Fragmentación externa, predeclaración de tamaño
- Necesidad de compactación

Fragmentación externa (ejemplo)

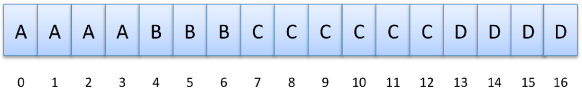
- Se desea crear un fichero de 4 bloques (D) en un SF con organización contigua



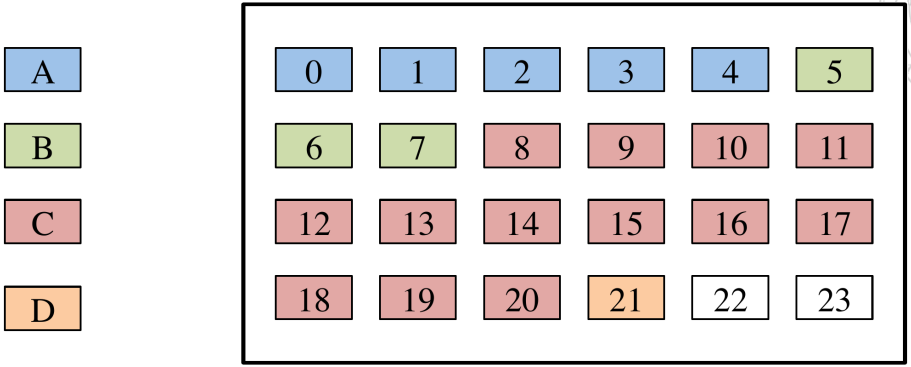
- 3 ficheros usan 13 de los 17 bloques disponibles
  - No es posible crear el fichero D en este escenario
- Aplicar compactación



- Crear el fichero

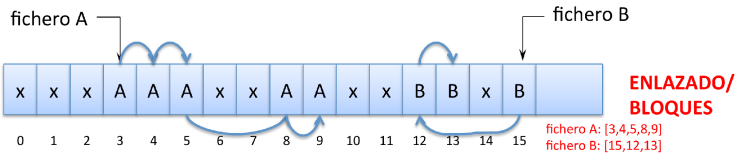


Organización contigua: CD-ROM (ISO-9660)



Organización enlazada (I)

- Se emplea una lista enlazada de bloques para representar un fichero
- La lista puede ser de dos tipos:
  - Lista de bloques
  - Lista de grupos de bloques



Organización enlazada (II)

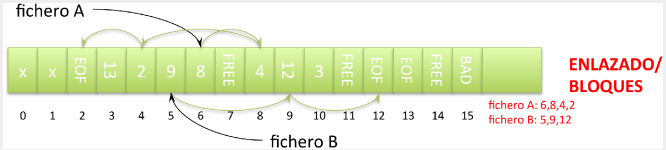
¿Dónde se almacenan los punteros de la lista?

- **Opción 1**: Dejar espacio al final de cada bloque de disco
  - Opción mala para acceso aleatorio
    - Acceso al bloque lógico n→Exige lectura de *n*bloques
  - Espacio para datos en un bloque $\neq 2^k$
- **Opción 2**: Dedicar una sección del disco aparte para almacenar los punteros
  - Estrategia utilizada en sistema de ficheros FAT

Organización enlazada (III)

File allocation table (FAT)

- Tabla que se almacena en una región especial de disco
  - Tiene tantas entradas como bloques direccionables tiene el disco
- Cada posición *i* de la tabla puede almacenar distintos valores:
  - Número: Puntero al siguiente bloque de lista enlazada
  - EOF: Indicador de fin de lista de bloques
  - FREE: Bloque i-ésimo está libre
  - BAD: Bloque i-ésimo está defectuoso



Organización enlazada (IV)

- Para localizar el byte *n*-ésimo de un fichero:
  - Calcular el bloque lógico que lo contiene: B = n/TamañoBloque
  - Leer la entrada de directorio para obtener el identificador del **primer** bloque del fichero
  - Seguir la cadena de punteros en la FAT hasta alcanzar el bloque lógico *B* (lo que cuesta *B-1* saltos)
  - Leer el bloque físico indicado por la última entrada de la FAT consultada
- Esta es la razón por la que el acceso aleatorio es costoso en un esquema enlazado: leer el byte 0 cuesta una consulta, pero leer un byte cercano al *final* de un fichero grande cuesta recorrer casi toda la cadena.

Organización enlazada (IV)

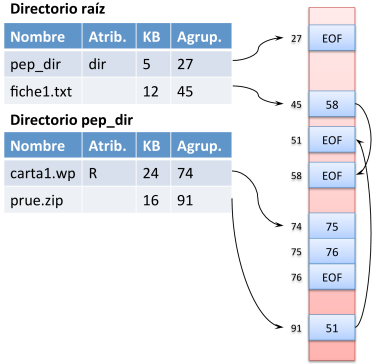
Ventajas

- No produce fragmentación externa
- Asignación dinámica simple
  - Cualquier bloque/grupo libre puede ser añadido al final de la lista de bloques
- Acceso secuencial fácil

Desventajas

- Accesos aleatorios requieren realizar un recorrido de la lista de bloques
- No toma en cuenta el principio de localidad
  - Los bloques de un fichero pueden estar desperdigados por el disco
- Es conveniente realizar compactaciones periódicas

Organización enlazada: MS-DOS (FAT)

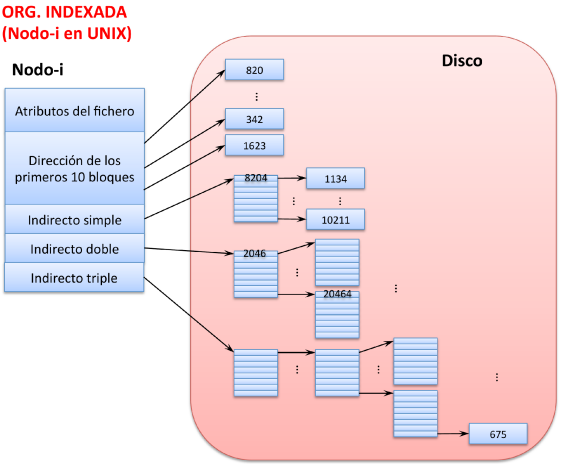


- Las distintas variantes de FAT usan agrupaciones (clusters) en lugar de bloques
- Una agrupación es un conjunto de bloques contiguos
- Cada fichero ocupa al menos una agrupación en disco

Organización indexada

- En la organización indexada, el descriptor físico del fichero contiene un conjunto de índices (punteros) para acceder a los datos
- Dos tipos de índices:
  - **Directo**: apunta a un bloque de datos
  - **Indirecto**: apunta a un bloque de índices
    - Cada bloque de índices contiene k punteros a bloques de datos o a otros bloques de índices
    - $k = \lfloor (\text{TamBloque(bytes)})/\text{TamIndice(bytes)} \rfloor$
    - Distintos tipos de bloques de índices según su nivel de indirección: simples, dobles, triples, ...

Organización indexada: Nodos-I (UNIX)



Encontrar el byte n-ésimo (I)

- `log_b` es el número de bloque lógico
- `m` es cuántos punteros caben en un bloque de índice

bloque_n-esimo.cpp

```
#define DP 12                // Punteros directos
#define BLOCK_SIZE 1024      // Tamaño de bloque
// n: posición del byte
block_id get_bid(int n, struct inode *p_inode) {
   int log_b = n/BLOCK_SIZE, m = BLOCK_SIZE/sizeof(block_id);
   int b_id, ind;
   block_id *p_block;
   if (log_b < DP) {                        // Directo
      b_id = p_inode->direct[log_b];
   } else if (log_b < (DP + m)) {           // Simple
      p_block = get_disk_block(p_inode->ind_simple);
      b_id = p_block[log_b - DP];
   } else if (log_b < (DP + m + m*m)) {     // Doble
      log_b = log_b - (DP + m);
      p_block = get_disk_block(p_inode->ind_double);
      ind = log_b / m;
      p_block = get_disk_block(p_block[ind]);
      ind = log_b % m;
```

Encontrar el byte n-ésimo (II)

- Nótese que el coste *no* es uniforme: un bloque directo cuesta una lectura de disco, pero un bloque alcanzado mediante indirección triple cuesta cuatro (tres bloques de índice más el bloque de datos)
- Por eso los inodos mantienen un conjunto de punteros directos para el caso común de ficheros pequeños.

bloque_n-esimo.cpp

```
    ...
    } else {                                // Triple
        log_b = log_b - (DP + m + m*m);
        p_block = get_disk_block(p_inode->ind_triple);
        ind = log_b / (m*m);
        p_block = get_disk_block(p_block[ind]);
        ind = (log_b % (m*m)) / m;
        p_block = get_disk_block(p_block[ind]);
        ind = (log_b % (m*m)) % m;
        b_id = p_block[ind];
    }
    return b_id;
}
```

El inodo: ¿qué almacena?

- El inodo (`index node`) es la estructura que almacena los metadatos de un fichero. Todo excepto su nombre y sus datos:

Campo	Significado
Dispositivo (major, minor)	Dispositivo que almacena el fichero
Número de inodo	Id. único en el sistema de ficheros
Propietario (UID) Grupo (GID)	Id. del usuario y grupo propietarios
Número de enlaces	Número de enlaces duros al fichero
Tamaño	Tamaño en bytes
Número de bloques	Número de bloques de 512 bytes
Marcas de tiempo	Creación, último acceso,...
Punteros de datos	Punteros a los bloques de los datos

El inodo en la práctica

- Todo atributo de la tabla del inodo se puede observar directamente con el comando `stat(1)`

Terminal #1

```
user@lab:~$ echo 'a' > file\.txt
user@lab:~$ stat file.txt
  File: file.txt
  Size: 2           Blocks: 8          IO Block: 4096   regular file
Device: 10302h/66306d   Inode: 19060945    Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1001/  carlos)   Gid: ( 1001/  carlos)
Access: 2026-09-14 22:58:00.517737505 +0200
Modify: 2026-09-14 22:58:00.517737505 +0200
Change: 2026-09-14 22:58:00.517737505 +0200
 Birth: 2026-09-14 22:58:00.517737505 +0200
```

- El contenido del fichero son 2 bytes (`a+\n`), pero ocupa **8x512 B = 4096 B** de espacio en disco: un bloque completo del tamaño de bloque de 4 KiB de este sistema de ficheros.
- Este desajuste entre el tamaño lógico (`Size:`) y el espacio asignado (`Blocks:`) significa que 4094 de esos bytes quedan sin usar, reservados pero nunca escritos. El fichero nunca puede ocupar menos de un bloque completo.

Permisos

- Cada fichero tiene una terna de permisos para tres sujetos: **propietario**, **grupo**, **otros**
- Cada sujeto tiene tres permisos posibles:
  - `r`: lectura
  - `w`: escritura
  - `x`: ejecución (o, en un directorio, permiso para hacer `cd` a él)
- Los permisos forman parte del campo *mode* del inodo, y son lo que `chmod(1)` modifica.
- Más allá de `rwx`, el campo mode de un fichero *ejecutable* puede llevar dos bits especiales:
  - **setuid**: el UID efectivo del proceso pasa a ser el UID del propietario del fichero mientras se ejecuta
  - **setgid**: el GID efectivo del proceso pasa a ser el grupo del fichero

Asignación indexada de ficheros

Ventajas

- No es necesario usar bloques de índices para ficheros pequeños *nodo-ialmacenatodalainformacin*
- Acceso secuencial y aleatorio sencillo
- No es preciso recorrer listas enlazadas

Desventajas

- Nodo-i debe estar en memoria para garantizar accesos eficientes
- Accesos aleatorios tienen tiempo de acceso variable
  - El coste depende del nivel de índice asociado al bloque lógico al que accedemos
- Tamaño máximo de fichero limitado por índices multinivel;
  - T_b: tamaño bloque; d: número de índices directos; n: tam. índice
  - $TamFicheroMax = (d + (T_b/n) + (T_b/n)^2 + (T_b/n)^3) \cdot T_b$

Ejemplo: ext2

Terminal #1

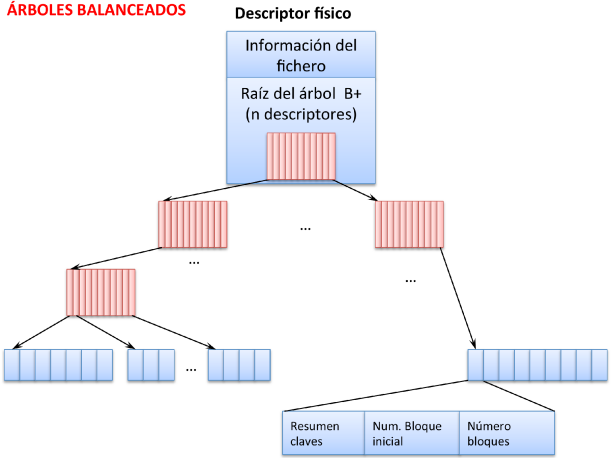
```
user@lab:~$ dd if=/dev/zero of=/tmp/disk.img bs=1024 count=100K
102400+0 records in
102400+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0,333543 s, 314 MB/s

user@lab:~$ mkfs -t ext2 -b 1024 /tmp/disk.img
Creating filesystem with 102400 1k blocks and 25688 inodes
...
user@lab:~$ sudo mount -t ext2 -o defaults,loop /tmp/disk.img /tmp/disk
user@lab:~$ cd /tmp/disk
user@lab:~$ sudo dd if=/dev/urandom of=file.bin bs=1024 count=1 seek=0
user@lab:~$ stat file.bin
  Size: 1024        Blocks: 2        IO Block: 1024   regular file
  Inode: 12         Links: 1
```

- Una escritura de 1 KiB ocupa 2 bloques de 512 bytes. Sin sorpresas por ahora, este fichero no tiene huecos

Organización basada en árboles equilibrados

ÁRBOLES BALANCEADOS



Gestión del espacio libre

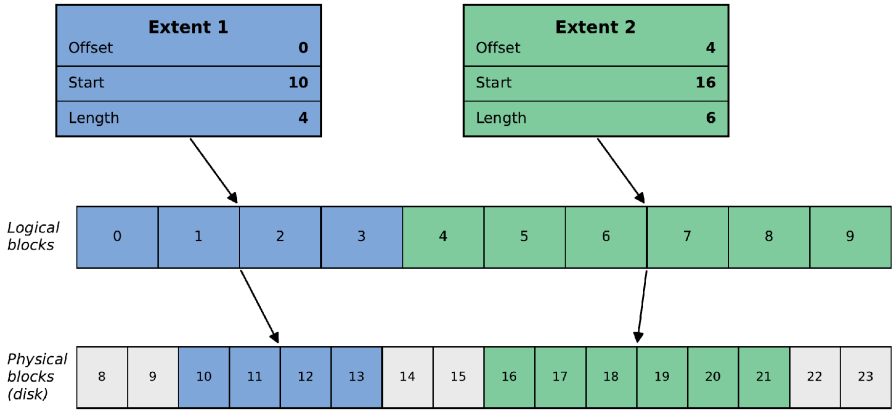
- El SO debe saber qué bloques y descriptores físicos de ficheros (p.ej., nodos-i) están libres
- Alternativas:
  - Mapa de bits (ej: 11000000100101010101001…)
    - Tamaño mapa(bytes) = tam_disco(bytes) / (8 * tam_bloque(bytes))
    - Ejemplo: 16 GB / (8 * 1KB) = 2 MB
  - Bloques libres encadenados
    - Lista de bloques libres implementada como pila o cola con parte en memoria
    - Variante: Lista de grupos de bloques libres (bloque-inicial, num-bloques)
  - Indexación de bloques libres
    - El espacio libre modelado como un fichero con organización indexada

Extents: la idea básica (I)



- Un **extent** es un grupo de bloques *contiguos*, descrito mediante sólo tres números en lugar de una entrada por bloque:
  - **Offset**: el número de bloque lógico del primer bloque del extent
  - **Start**: el número de bloque físico donde empieza el extent
  - **Length**: cuántos bloques abarca el extent
- Este es un término medio entre la asignación contigua y la asignación indexada (un puntero por *bloque*): un fichero se describe mediante unos pocos segmentos (regiones contiguas de bloques), lo cual es compacto **siempre que el fichero no esté demasiado fragmentado**

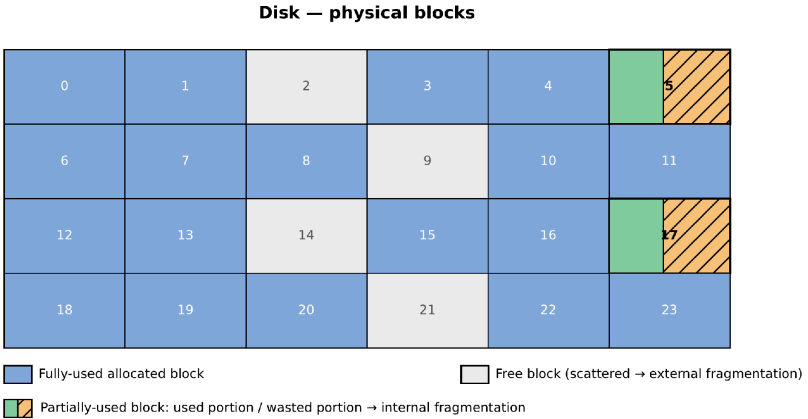
Extents: la idea básica (II)



Extents: localidad y fragmentación (I)

- **Localidad espacial:** mantener juntos los bloques de un fichero mejora el rendimiento:
  - Discos mecánicos: menos movimiento del cabezal/búsqueda
  - SSDs: operaciones de E/S menos numerosas y más grandes, mejor gestión interna del espacio
- Dos problemas distintos, ambos llamados "fragmentación":
  - **Fragmentación externa**: el espacio libre queda dividido en pequeños huecos no contiguos repartidos por el disco
  - **Fragmentación interna**: espacio desperdiciado *dentro* de un bloque asignado que solo se usa parcialmente
- Estrategias que usan los sistemas de ficheros modernos para mitigar ambas:
  - **Extents** en lugar de indexación por bloque
  - **Asignación retardada**: los bloques físicos solo se asignan cuando los datos realmente necesitan volcarse de la page cache al disco
  - **Preasignación**: asignar más bloques de los solicitados inmediatamente (p. ej., el asignador multibloque de ext4)

Extents: localidad y fragmentación (II)



Extents en disco: el formato de ext4

- Un inodo de ext4 almacena hasta **4 extents directamente**
- No hace falta ningún árbol hasta que un fichero necesita un 5º extent
- Cada estructura de extent ocupa 12 bytes; por tanto, un bloque de 4 KiB puede contener aproximadamente $4096/12 \approx 340$ extents
- Un extent individual tiene un límite máximo de longitud fijo (p. ej., $2^{15} \cdot 4Kblocks = 128MiB$ en ext4)

Extents: fichero pequeño (I)

- Ejemplo: un fichero formado por cuatro secciones de 3 KiB, cada una escrita a 1 MiB de distancia (de forma que cada sección se convierte en un extent independiente de 1 bloque)

Terminal #1

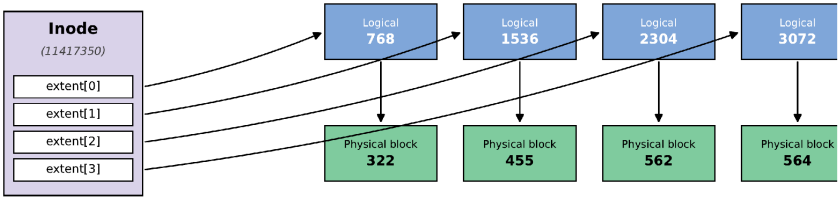
```
user@lab:~$ $ sudo debugfs -R "dump_extents <11417350>" /dev/sda1

Level Entries       Logical     Physical Length
 0/ 0   1/  4   768 -   768  322 -  322      1
 0/ 0   2/  4  1536 -  1536  455 -  455      1
 0/ 0   3/  4  2304 -  2304  562 -  562      1
 0/ 0   4/  4  3072 -  3072  564 -  564      1
```

- `Level 0/0` significa que estos 4 extents residen directamente en el inodo. Como se describió antes, todavía sin árbol de índice

Extents: fichero pequeño (II)

Small file: 4 extents, stored directly in the inode (Level 0/0)

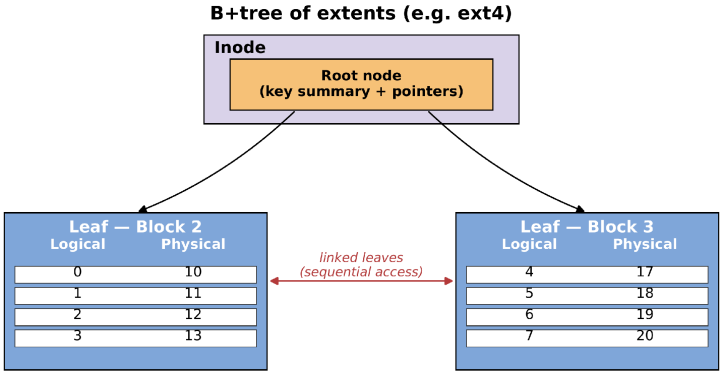


Level 0/0 → all 4 extents live directly in the inode, no index tree needed yet — each extent has Length = 1

B+trees de extents (I)

- Cuando un fichero necesita más extents de los que caben directamente en el inodo, estos se organizan en un **B+tree**, en lugar de en el índice multinivel:
  - El inodo contiene el **nodo raíz** del árbol (además de los metadatos del fichero, como siempre)
  - Los nodos internos almacenan punteros a los siguientes niveles
  - Los **nodos hoja** están enlazados entre sí, de forma que un recorrido puramente secuencial del fichero nunca tiene que volver a subir por el árbol
- En ext4, el árbol puede llegar a tener hasta 5 niveles de profundidad; esto proporciona un acceso eficiente tanto para ficheros muy grandes como muy fragmentados, razón por la cual los sistemas de ficheros de propósito general modernos (ext4, XFS, NTFS, JFS, ...) han convergido todos hacia este enfoque

B+trees de extents (II)



For small files, the root node stores extents directly — no leaf blocks are needed.

Extents en la práctica: cuando el fichero crece

- Añadimos dos secciones más de 3 KiB (2 extents más) al fichero del ejemplo anterior

Terminal #1

```
user@lab:~$ sudo debugfs -R "dump*extents <11417350>" /dev/sda1
Level Entries       Logical     Physical Length
 0/ 1   1/  1   768 -  4608  1273         3841
 1/ 1   1/  6   768 -   768  322 -  322      1
 1/ 1   2/  6  1536 -  1536  455 -  455      1
 1/ 1   3/  6  2304 -  2304  562 -  562      1
 1/ 1   4/  6  3072 -  3072  564 -  564      1
 1/ 1   5/  6  3840 -  3840  666 -  666      1
 1/ 1   6/  6  4608 -  4608  677 -  677      1
```

- El inodo (`Level 0`) ya no almacena extents directamente. Ahora almacena **un único puntero a un bloque hoja** (bloque físico 1273), que a su vez contiene los 6 extents (`Level 1`).

Extents: un fichero mucho más grande

- Tras hacer crecer el mismo fichero hasta un total de 341 bloques, el árbol se ha dividido más: la implementación de ext4 divide un nodo en el punto de inserción en cuanto se llena.

Terminal #1

```
Root node (in the inode):        pointers → block 1273,  block 4533
Leaf node (in block 1273):       covers logical range   768 - 261887
New leaf node (in block 4533):   covers logical range 261888 - 261888
```

Ficheros dispersos (sparse): el concepto

- Un **fichero disperso (sparse file)** es un fichero con regiones que nunca se han escrito, "huecos", para las que el sistema de ficheros no almacena (ni necesita leer) ningún bloque de disco real
- La implementación depende del esquema de asignación:
  - **Asignación indexada:** solo se inicializan las entradas de índice de los bloques que realmente se han escrito; el resto se dejan como punteros nulos
  - **Extents:** simplemente no existe un extent para un hueco: no hay nada que almacenar
- Ejemplo (extents): un fichero de 1 MiB en la posición lógica 100 MiB:

```
Terminal #1
[Extent 0] Offset: 102400  Length: 1024  Start: 21504
```

sólo la región escrita (bloques lógicos 102400–103423) tiene un extent real; los 100 MiB anteriores no se corresponden con ningún extent.

Los ficheros dispersos en la práctica

Terminal #1

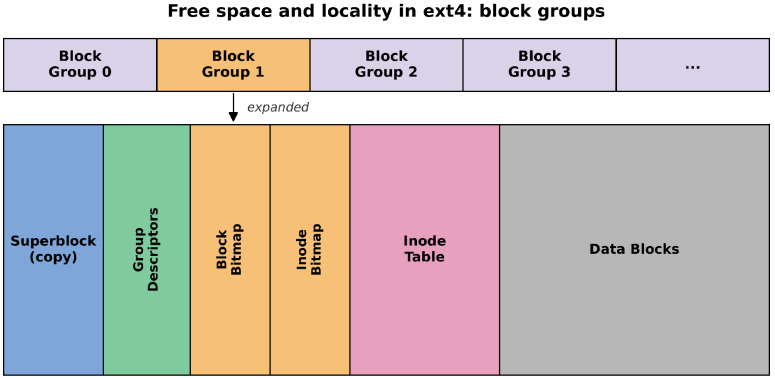
```
user@lab:~$ dd if=/dev/zero of=sparse.img bs=1M seek=100 count=1
user@lab:~$ ls -slh sparse.img
 1.0M -rw-r--r-- 1 user *OS user *OS 101M Feb 3 10:38 sparse.img
```

- `seek=100` le indica a 'dd que empiece a escribir 100 MiB *dentro* del fichero sin tocar nada antes de ese punto
- El resultado: un fichero cuyo **tamaño lógico es 101 MiB**, pero que solo ocupa **1 MiB de espacio real en disco**
- `ls -sl` muestra ambos valores
- Un fichero disperso es una característica general de los sistemas de ficheros, no un característica exclusiva de ext4

Espacio libre en ext4 (I)

- 'ext4 divide el disco en **grupos de bloques**, cada uno una región autocontenida con sus propios metadatos:
  - Superbloque (copia)
  - Descriptores de grupo
  - Mapa de bits de bloques
  - Mapa de bits de inodos
  - Tabla de inodos
  - Bloques de datos
- **Localidad:** mantener el inodo de un fichero, el inodo de su directorio y sus bloques de datos dentro del *mismo* grupo de bloques hace que se reduzca la distancia de búsqueda, mejorando los patrones de acceso
- El mapa de bits de cada grupo de bloques marca si cada inodo/bloque de ese grupo está libre; también existen estructuras alternativas, p. ej., XFS

Espacio libre en ext4 (II)



Bitmaps mark which inodes/blocks in this group are free. Keeping a file's inode, its directory, and its data blocks in the same group keeps related I/O physically close together.

Concepto de directorio

- Objeto que relaciona de forma unívoca un nombre de fichero (descriptor lógico) con su descriptor interno o físico
- Se representa típicamente como una tabla (conjunto de entradas de directorio)
- En algunos sistemas, como las distintas variantes de FAT, la entrada de directorio es en sí el descriptor físico del fichero
- Los directorios también permiten al usuario organizar su información de forma estructurada
- Un directorio es un nodo en sistemas de ficheros jerárquicos

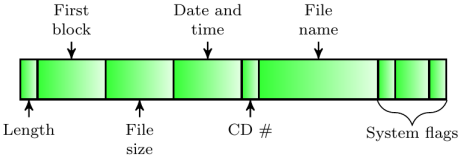
Propiedades deseables de los directorios

- **Eficiencia**: localizar un fichero rápidamente
- **Sencillez**: la entrada de directorio debe ser lo más sencilla posible
- **Nombrado**: conveniente y sencillo para los usuarios
  - Dos usuarios pueden tener el mismo nombre para ficheros distintos
  - Un mismo fichero puede tener nombres distintos
  - Nombres de longitud variable
- **Agrupación**: agrupación lógica de los ficheros según sus propiedades (por ejemplo: programas C, juegos, etc.)
- **Visión estructurada**: operaciones claramente definidas y ocultación

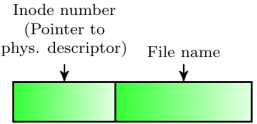
Estructura de los directorios

- Tanto la estructura del directorio como los ficheros residen en discos
- Los directorios se suelen implementar como ficheros con formato conocido para el SO
- Se han explorado principalmente dos implementaciones de directorio:
- Cada entrada en un directorio es el descriptor físico del fichero
  - Los atributos del fichero se almacenan en su entrada del directorio
  - Ejemplo: CD-ROM, FAT
- Directorio es simplemente una tabla de pares (*nombre_fichero*, *ID*)
  - ID es un identificador único ("puntero") para referirse al descriptor físico del fichero
  - Ejemplo: UNIX

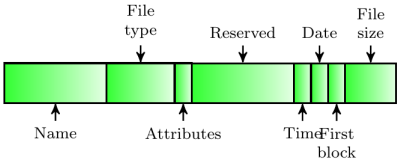
Contenido de la entrada de directorio



Entrada de directorio en ISO-9660 (CD-ROM)



Entrada de directorio en Unix SV



Entrada de directorio en FAT (MS-DOS)

¿Dónde se almacena el mapa nombre-inodo?

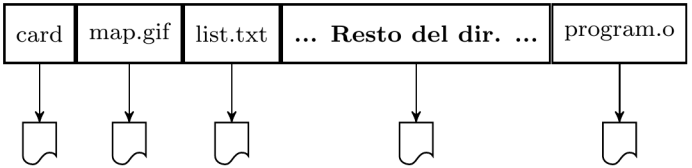
- En los sistemas de ficheros de estilo UNIX, una entrada de directorio es simplemente un par (nombre, número de inodo). El resto de atributos residen en el propio inodo.
- Dependiendo de cuántas entradas tenga un directorio, ese mapa nombre→inodo se puede almacenar:
  - directamente dentro del propio inodo del directorio (directorios muy pequeños)
  - en un único bloque de datos
  - repartido en varios bloques de datos, accesibles mediante la misma asignación indexada/basada en extents que se usa para los ficheros regulares. Al fin y al cabo, un directorio no es más que un tipo especial de fichero

Permisos de directorio y el sticky bit

- El modelo de permisos `rwx` (campo mode del inodo) también se aplica a los directorios, pero ahí los tres bits significan algo distinto:
  - `r`: listar los nombres del directorio
  - `w`: crear o eliminar entradas *dentro* del directorio (no editar su contenido)
  - `x`: atravesar el directorio (p. ej., `cd`, o resolver una ruta a través de él)
- Dos bits especiales son propios de los directorios:
  - **setgid** (en un directorio): las nuevas entradas creadas dentro heredan el grupo *del directorio*, en lugar del grupo del proceso que las crea
  - **sticky bit** (en un directorio): las entradas solo pueden ser eliminadas o renombradas por su propietario, por el propietario del directorio, o por un proceso privilegiado. El ejemplo clásico es `/tmp`, que debe seguir siendo escribible por todos pero no debe permitir que un usuario elimine los ficheros de otro

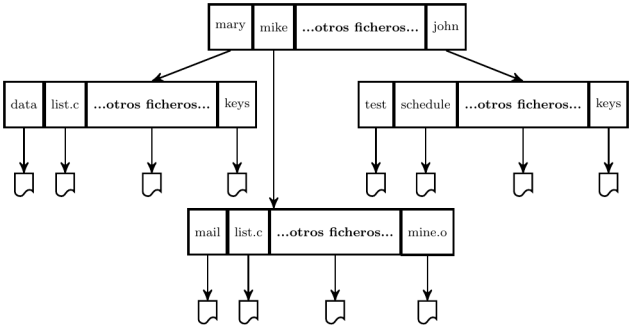
Directorio de un nivel

- Un único directorio en el sistema
  - Compartido por todos los usuarios
- Problemas de nombrado y agrupación



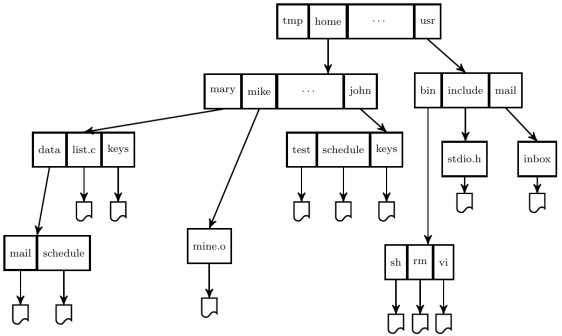
Directorio de dos niveles

- Un directorio por cada usuario
- Caminos de acceso pueden ser explícitos (*absolutos*) o implícitos (relativos al directorio del usuario)
- Varios usuarios pueden poseer ficheros distintos con el mismo nombre
- Búsqueda eficiente, pero problemas de agrupación



Directorio con estructura de árbol

- Búsqueda eficiente y agrupación
- Necesario mecanismos más sofisticados para referirse a los ficheros en la jerarquía
  - Nombres relativos y absolutos
  - Directorio de trabajo



Nombres jerárquicos (I)

Directorios especiales

- **Directorio de trabajo "."**
  - Ejemplo: `cp /users/mike/keys .`
  - En BASH se cambia con `cd`, y la ruta se obtiene con `pwd`
- **Directorio padre ".."**
  - Ejemplo: `ls ..`
- **Directorio HOME**: El directorio *base* del usuario
  - `cd` o `cd $HOME`

SO	Ubicación por defecto del HOME
Windows 7 y posteriores	`C:\Users\<username>`
GNU/Linux	`/home/<username>`
Solaris	`/export/home/<username>`
Mac OS X	`/Users/<username>`

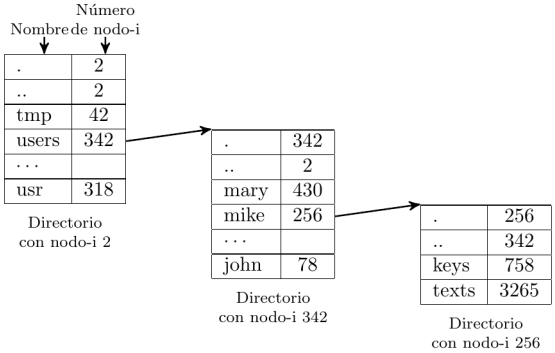
Nombres jerárquicos (II)

Dos tipos de ruta

- **Ruta absoluta**: especificación del nombre con respecto al directorio raíz ("/" en LINUX, "\" en Windows)
  - Ejemplo: `/users/mike/keys`
- **Ruta relativa**: especificación del nombre con respecto al directorio de trabajo del proceso
  - Relativos al dir. de trabajo o actual: aquel en el se está al indicar el nombre relativo.
  - Ejemplo: (Dir. actual es `/users`)
    - `mike/keys`
    - `../users/mike/keys`
    - `./mike/keys`

Interpretación de nombres en Linux (I)

- Obtener el descriptor físico del fichero a partir de una ruta
- Ejemplo para `/users/mike/keys`



Búsqueda rápida en directorios grandes (II)

- Ejemplo: Buscar el fichero `shadow` en `/etc` en un sistema de ficheros ext4.

1. Calcular el hash del nombre del archivo
2. Realizar la búsqueda binaria en el árbol
3. Buscar el bloque el nombre
4. Calcular el hash del nombre del archivo
5. Realizar la búsqueda binaria en el árbol
6. Buscar el bloque el nombre

Terminal #1

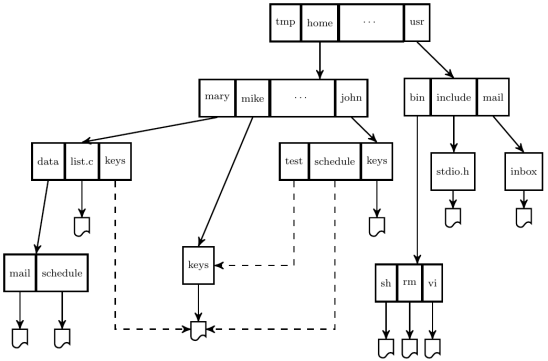
```
user@lab:~$ sudo debugfs -R "dx*hash shadow" /dev/sda1
Hash of shadow is 0xbfa46a3c

user@lab:~$ sudo debugfs -R "htree etc" /dev/sda1
Root node dump:
  Number of entries (count): 4
Entry #0: Hash 0x00000000, block 1
Entry #1: Hash 0x4b855aee, block 3
Entry #2: Hash 0x8a3857da, block 2
Entry #3: Hash 0xc0ba6778, block 4


user@lab:~$ ls -il /etc/shadow
1486 -r-------- 1 root root 554 Sep 25 18:45 /etc/shadow
```

Directorio de grafo acíclico

- Dos rutas no equivalentes pueden llevarnos al mismo fichero
- El sistema de ficheros debe soportar la creación de *alias*



Jerarquías de grafo acíclico: enlaces (I)

- **Enlace**: una de los posibles nombres para referirse a un archivo
  - **Físico o rígido**: Un único fichero con contador enlaces en descriptor
    - Número de alias del fichero en el sistema
  - **Simbólico**: Un fichero especial que almacena la ruta de otro fichero
    - Se dice que un enlace simbólico está *roto* si la ruta que almacena no existe
- Posibles estrategias de borrado de enlaces:
  - (enlaces rígidos): Decrementar contador de enlaces; si 0 $\rightarrow$ borrar fichero
  - Recorrer los enlaces y borrar todos
  - (enlaces simbólicos): Borrar únicamente el enlace y dejar los demás

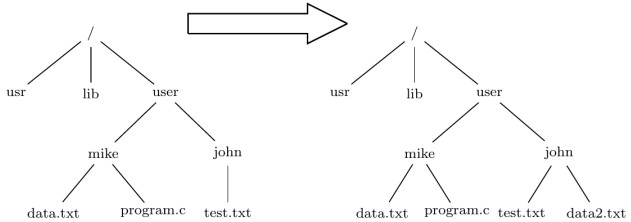
Jerarquías de grafo acíclico: enlaces (II)

Problemas y limitaciones

- La creación de enlaces puede conllevar la creación de ciclos en el grafo. Dos soluciones:
  - Permitir sólo enlaces a ficheros, no a directorios
  - Algoritmo de búsqueda de ciclos cuando se solicita la creación de un enlace
- En UNIX, solo se permite crear enlaces físicos dentro del mismo sistema de ficheros
  - Enlaces simbólicos sí permitidos entre sistemas de ficheros

Enlace físico: ejemplo

```
$ ln /user/mike/data.txt /user/john/data2.txt
```



mike

.	23
..	100
data.txt	28
program.c	400

john

.	80
..	100
test.txt	60

mike

.	23
..	100
data.txt	28
program.c	400

john

.	80
..	100
test.txt	60
data2.txt	28

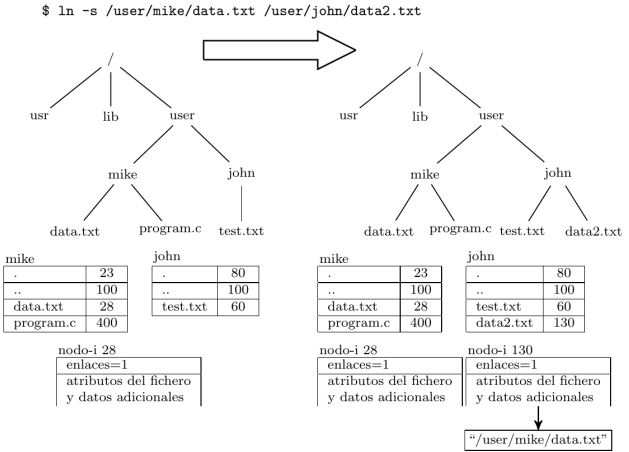
nodo-i 28

enlaces=1
atributos del fichero y datos adicionales

nodo-i 28

enlaces=2
atributos del fichero y datos adicionales

Enlace simbólico: ejemplo



Enlaces físicos vs simbólicos

Terminal #1

```
user@lab:~$ ls
info.pdf
user@lab:~$ ln info.pdf alias1
user@lab:~$ ln -s info.pdf alias2
user@lab:~$ stat info.pdf
  File: ‘info.pdf’
  Size: 68883       Blocks: 136        IO Block: 1048576 regular file
Device: 24h/36d Inode: 26802399    Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1058/  user)   Gid: ( 1060/  user)
Access: 2015-08-31 14:36:43.564966921 +0200
Modify: 2015-08-31 14:36:43.585657807 +0200
Change: 2015-08-31 14:37:00.532906762 +0200
```

- Se crean dos enlaces al fichero `info.pdf`
  - alias1 (enlace físico)
  - alias2 (enlace simbólico)
- El comando `stat` imprime por pantalla los atributos de un ficheros, así como el número de nodo-i y el valor actual del contador de enlaces

Enlaces físicos vs simbólicos

Terminal #1

```
user@lab:~$ stat alias1
  File: ‘alias1’
  Size: 68883       Blocks: 136        IO Block: 1048576 regular file
Device: 24h/36d Inode: 26802399    Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1058/  user)   Gid: ( 1060/  user)
Access: 2015-08-31 14:36:43.564966921 +0200
Modify: 2015-08-31 14:36:43.585657807 +0200
Change: 2015-08-31 14:37:00.532906762 +0200
 Birth: -
user@lab:~$ stat alias2
  File: ‘alias2’ -> ‘info.pdf’
  Size: 8           Blocks: 1          IO Block: 1048576 symbolic link
Device: 24h/36d Inode: 26802400    Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1058/  user)   Gid: ( 1060/  user)
Access: 2015-08-31 14:37:06.702552995 +0200
Modify: 2015-08-31 14:37:06.702552995 +0200
Change: 2015-08-31 14:37:06.702552995 +0200
 Birth: -
```

- `info.pdf` y `alias1` “comparten” el mismo nodo-i
- ‘alias2 no es un fichero regular y usa un nodo-i diferente

Sistemas de ficheros

- El sistema de ficheros permite organizar la información dentro de los dispositivos de almacenamiento secundario en un formato inteligible para el SO
- Previamente a la instalación del SF, es necesario dividir físicamente, o lógicamente, los discos en particiones o volúmenes
  - Una partición es una porción de un disco a la que se la dota de una identidad propia y que puede ser manipulada por el SO como una entidad lógica independiente
  - Estrategias de particionado típicas:
    - Master Boot Record (MBR)
    - GUID/GPT - Mac OS X (x86)
    - Apple Partition Map (APM) - Mac OS (PowerPC)
- Una vez creadas las particiones, el SO debe crear las estructuras de los SF dentro de esas particiones
  - Para ello se proporcionan al usuario comandos como `format` (Windows) o `mkfs` (UNIX)

Metadatos de una partición

- El sistema de ficheros instalado en una partición ha de ser autocontenido
  - Tanto datos como metadatos del SF se almacenan en disco

Tipos de metadatos

1. Parámetros globales del SF
   - Tamaño de la partición
   - Tamaño de bloque
   - Número de bloques de datos y de descriptores físicos
2. Metadatos críticos
   - Directorio raíz
   - Tablas de índices (en FAT) o array de descriptores físicos (nodos-i en UNIX)
3. Estructuras para gestión del espacio libre
   - Control de bloques del disco libres/ocupados
   - Control de descriptores físicos libres/ocupados (p.ej., nodos-i)

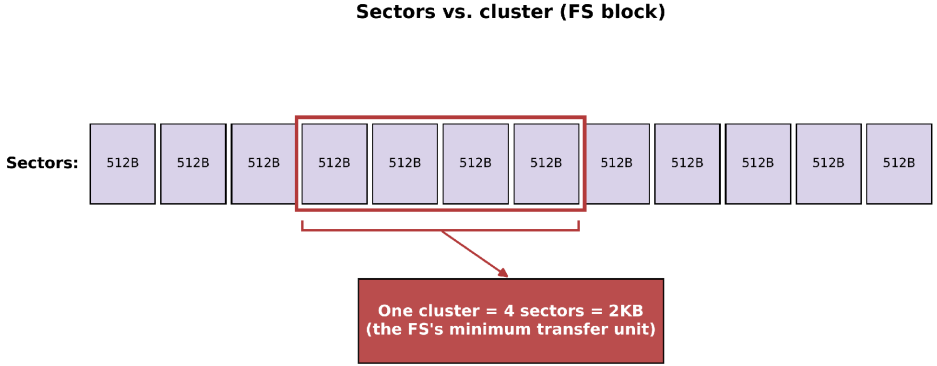
Particiones y volúmenes

- Una **partición** es una porción de un disco a la que se da identidad propia, que el SO puede manipular como una entidad lógica independiente
- Un **volumen** es un conjunto de particiones de disco que el SO puede tratar como una única unidad lógica
  - Un volumen puede abarcar varios discos físicos. Así es como el SO soporta cosas como RAID o los gestores de volúmenes lógicos (LVM)
- Los sistemas de ficheros se crean sobre particiones o volúmenes, no directamente sobre un disco físico en bruto
- Una vez que existen las particiones/volúmenes, el SO todavía tiene que construir dentro de ellas las estructuras propias del sistema de ficheros (`format`/`mkfs`)

Sector y clúster (I)

- **Sector**: la unidad mínima de transferencia que el controlador de disco puede manejar, 2^m bytes (normalmente 512 bytes)
- **Clúster** (o bloque del SF): una agrupación lógica de sectores de disco que forma la unidad mínima de transferencia usada por el sistema de ficheros
  - Por tanto, un fichero ocupa, como mínimo, un clúster completo
  - Los sistemas de ficheros tienen un tamaño de clúster/bloque por defecto, pero normalmente se puede especificar uno distinto al ejecutar `mkfs`
  - Los clústeres más grandes mejoran la eficiencia de E/S en el almacenamiento secundario (menos transferencias, pero más grandes)
  - El coste a cambio es la **fragmentación interna**: el espacio que un fichero ocupa en disco siempre es un múltiplo del tamaño de clúster (diapositiva "Fragmentación interna", más adelante)

Sector y clúster (II)



A file smaller than 2KB still occupies one full cluster → internal fragmentation

Sistemas de ficheros y particiones

- Volumen o partición: conjunto coherente de metainformación y datos.
- Ejemplos:

CD-ROM



FAT (MS-DOS)

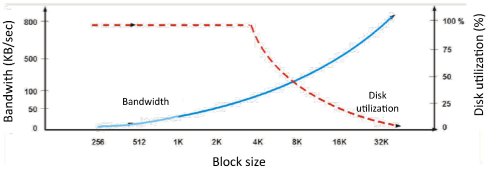


UNIX



Tamaño de bloque

- **Bloque**: agrupación lógica de sectores de disco y es la unidad de transferencia mínima que usa el sistema de ficheros.
  - Todo fichero ocupa al menos un bloque en disco
  - La elección del tamaño de bloque tiene un impacto importante en el rendimiento de la E/S
  - Todos los sistemas operativos definen un tamaño de bloque por defecto
  - Los usuarios pueden definir el tamaño de bloque a usar en el sistema de ficheros al instalarlo en una partición



Fragmentación interna

- Sistema de ficheros donde el volumen está completamente lleno de ficheros de 7KB cada uno
  - TBloque = 2KB
- Calcular el grado de fragmentación interna (fracción de disco desaprovechada) suponiendo despreciable el espacio ocupado por los metadatos del SF
  - ¿Cuánto espacio se necesita para almacenar un fichero de 7KB?
    - NBloques= $\lceil$ (TFichero(bytes)/Tbloque(bytes) $\rceil$ = $\lceil$ 7kB/2kB $\rceil$ = 4 bloques (8kB)
    - Porcentaje de espacio desaprovechado = (1KB)/(8KB) $\cdot$ 100=12.5 %
- Para un disco de 1TB $\rightarrow$ se desperdiciarían 125GB!!

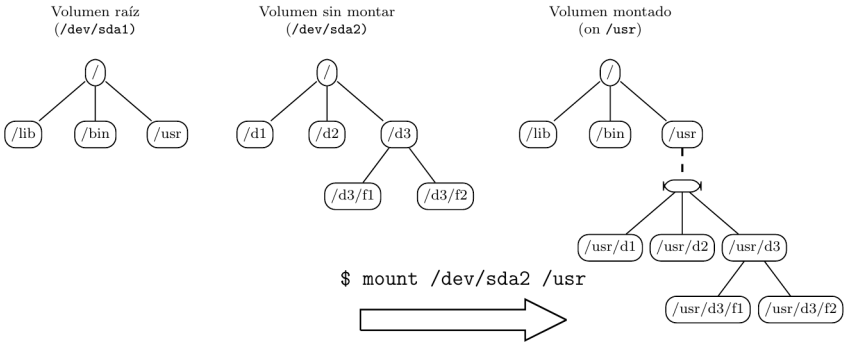


Gestión de múltiples sistemas de ficheros



- ¿Cómo exponer múltiples sistemas de ficheros al usuario?
  - En Windows se emplea un árbol de directorios distinto por cada sistema de ficheros (`c:\users\mike\keys`,`j:\joe\tmp`)
  - En UNIX/Linux se mantiene un árbol único para todo el sistema (`/users/mike/keys`, `/mnt/joe/tmp`, ...)
- En UNIX se proporcionan comandos de administración para mantener el árbol único: `mount`, `umount`
  - `mount /dev/sda3 /users`
  - umount /users‘
- Tener un árbol único proporciona una imagen unificada del sistema y oculta el tipo de dispositivo al usuario
- Lamentablemente, tener un único árbol complica la traducción de nombres, y origina problemas relacionados con los enlaces

Montaje de sistemas de ficheros o particiones



Interfaz de línea de comandos: ficheros

- Comandos para consultar y actualizar el estado de un fichero (su inodo):
  - `stat(1)`: muestra información sobre un fichero o sistema de ficheros
  - `touch(1)`: actualiza las fechas de acceso y modificación
  - `chown(1)`: cambia el propietario (usuario y/o grupo)
  - `chmod(1)`: cambia el modo (permisos)
  - `ln(1)`: crea enlaces duros y simbólicos
  - `readlink(1)`: lee la ruta almacenada en un enlace simbólico
- Otros comandos ya vistos:
  - `ls(1)`: lista el contenido de un directorio y los atributos de los ficheros
  - `rm(1)`, `cp(1)`, `mv(1)`: eliminan, copian y renombran ficheros
  - `mkdir(1)`, `rmdir(1)`: crean y eliminan directorios

Servicios POSIX para ficheros (I)

- Visión lógica
  - Fichero: secuencia de bytes
  - Puntero de posición a partir del cual se efectúan las operaciones
- Descriptores de ficheros: enteros de 0 a 64K
  - Valor entero que maneja el programador para realizar operaciones sobre el fichero abierto
  - Puede referirse a un fichero regular o a fichero especial
  - **Advertencia:** Descriptor de un fichero abierto (abstracción del API) $\neq$ descriptor lógico (nombre del fichero) $\neq$ descriptor físico o interno (p. ej., nodo-i)
- Descriptores numéricos predefinidos:
  - 0 $\rightarrow$ entrada estándar
  - 1 $\rightarrow$ salida estándar
  - 2 $\rightarrow$ salida de error

Servicios POSIX para ficheros

- Se proporcionan servicios para consultar y modificar el valor de los atributos de un fichero
- Cada fichero tiene información de protección asociada
- propietario (rwx), grupo(rwx), todos(rwx)
  - Los permisos de lectura (r), escritura (w) y ejecución (x) para cada *agente* del sistema se codifican con 3 bits
  - El API de ficheros representa esta información mediante 3 dígitos octales
    - Ejemplo: 755 significa *rwxr-xr-x*

La estructura stat

- `int stat(const char *pathname, struct stat *buf);`
- `int fstat(int fd, struct stat *buf);`
- `lstat()` es el equivalente para enlaces simbólicos

```
<sys/stat.h>
struct stat {
    dev_t     st_dev;     /* Dispositivo que contiene el fichero */
    ino_t     st_ino;     /* Número de inodo                     */
    mode_t    st_mode;    /* Tipo de fichero y permisos          */
    nlink_t   st_nlink;   /* Número de enlaces duros             */
    uid_t     st_uid;     /* UID del propietario                 */
    gid_t     st_gid;     /* GID del propietario                 */
    dev_t     st_rdev;    /* Dispositivo, si es fichero especial */
    off_t     st_size;    /* Tamaño en bytes                     */
    blksize_t st_blksize; /* Tamaño de bloque de E/S preferido */
    blkcnt_t  st_blocks;  /* Bloques físicos asignados           */
    time_t    st_atime;   /* Última fecha de acceso              */
```

Operaciones básicas sobre ficheros

- **creat**: crea un fichero con un nombre y protección y devuelve un descriptor
- **open**: abre un fichero con nombre para realizar operaciones sobre él y devuelve un descriptor
- **unlink**: borra el fichero con un nombre
- **read**: lee datos de un fichero abierto, usando su descriptor, copiándolos a una región de memoria del proceso
- **write**: escribe datos a un fichero abierto, usando su descriptor, desde una región de memoria del proceso
- **close**: cierra un fichero abierto con un descriptor
- **lseek**: mueve el puntero de posición del fichero (movimiento relativo a un punto dado)
- **stat**: devuelve los atributos de un fichero

Crear un fichero

creat()

- `int creat(char *name, mode_t mode);`
- Argumentos:
  - `name`: Nombre de fichero
  - `mode`: Bits de permiso para el fichero
- Valor de retorno:
  - Devuelve un descriptor de fichero ó -1 si error.
- Ejemplos

```
fd=creat("data.txt", 0751);
fd=open("data.txt", O_WRONLY|O_CREAT|O_TRUNC,0751);
```

Abrir un fichero

open()

- `int open(char *name, int flag, ...);`
- Argumentos:
  - `name`: ruta del fichero
  - `flags`: opciones de apertura
    - `O_RDONLY` Sólo lectura
    - `O_WRONLY` Sólo escritura
    - `O_RDWR` Lectura y escritura
    - `O_APPEND` El puntero de posición se desplaza al final del fichero
    - `O_CREAT` Si el fichero existe no tiene efecto. Si no existe lo crea
    - `O_EXCL` | usado con `O_CREAT`: genera un error si el fichero ya existe
    - `O_TRUNC` Trunca el fichero si se abre para escritura
- Valor de retorno:
  - Un descriptor de fichero ó -1 si hay error

Eliminar una entrada de directorio

`unlink()`

- `int unlink(const char* path);`
- Argumentos:
  - `path`: nombre del archivo
- Valor de retorno:
  - Devuelve 0 ó -1 si error.
- Descripción:
  - Elimina la entrada de directorio y decrementa el número de enlaces del archivo correspondiente
  - Cuando el número de enlaces es igual a cero y ningún proceso lo mantiene abierto, se libera el espacio ocupado por el fichero y el fichero deja de ser accesible

Leer de un fichero

read()

- `ssize_t read(int fd, void *buf, size_t n_bytes);`
- Argumentos:
  - `fd`: descriptor de fichero
  - `buf`: dirección de memoria de la región donde se copiarán los datos
  - `n_bytes`: número de bytes a leer
- Valor de retorno:
  - Número de bytes realmente leídos ó -1 si error
- Descripción:
  - Transfiere `n_bytes` (como mucho) al espacio de direcciones del proceso
    - Podrían transferirse menos de n_bytes (p.ej., se llegó a fin del fichero)
  - `read()` avanza de forma implícita el puntero de posición del fichero tantas posiciones cómo bytes se hayan transferido

Escribir en un fichero

`write()`

- `ssize_t write(int fd, void *buf, size_t n_bytes);`
- Argumentos:
  - `fd`: descriptor de fichero
  - `buf`: zona de datos a escribir
  - `n_bytes`: número de bytes a escribir
- Valor de retorno:
  - Número de bytes realmente escritos ó -1 si error
- Descripción:
  - Transfiere n_bytes (como mucho) al fichero
    - Podrían transferirse menos de n_bytes (p.ej., se llegó al tamaño máximo de un fichero o se interrumpe por una señal)
  - `write()` avanza de forma implícita el puntero de posición del fichero tantas posiciones cómo bytes se hayan transferido

Cerrar un fichero

close()

- `int close(int fd);`
- Argumentos:
  - `fd`: descriptor de fichero
- Valor de retorno:
  - Devuelve 0 ó -1 si error
- Descripción:
  - El proceso pierde la asociación a un fichero

Modificar el puntero de posición (I)

lseek()

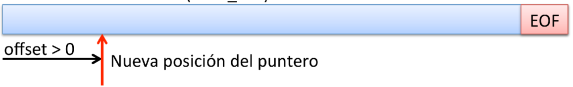
- `off_t lseek(int fd, off_t offset, int whence);`
- Argumentos:
  - `fd`: Descriptor de fichero
  - `offset`: desplazamiento relativo or absoluto
  - `whence`: base del desplazamiento
- Valor de retorno:
  - La nueva posición del puntero ó -1 si error
- Descripción:
  - Establece una nueva ubicación del puntero de posición del fichero:
    - `whence == SEEK_SET` → posición = offset
    - `whence == SEEK_CUR` → posición = posición actual + offset
    - `whence == SEEK_END` → posición = tamaño fichero + offset

Modificar el puntero de posición (II)

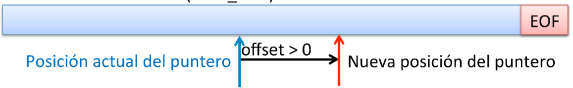
```
off_t lseek(int fd,
            off_t offset,
            int whence)
```

En un sistema de ficheros tipo Unix, es sencillo realizar un lseek debido a la estructura lineal de los ficheros. De hecho, esta operación se realiza habitualmente con una única búsqueda en disco.

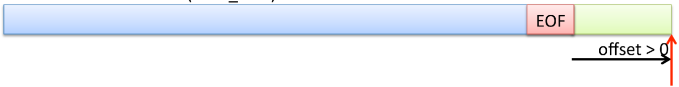
- Caso 1: whence == 0 (SEEK_SET)



- Caso 2: whence == 1 (SEEK_CUR)



- Caso 3: whence == 2 (SEEK_END)



Sincronizando con disco: fsync()

fsync()

- `int fsync(int fd);`
- Como se indicó en la diapositiva de `write()`, las escrituras normalmente solo llegan a la page cache. El SO las vuelca a disco más tarde, según su propia planificación (las políticas de escritura de la caché que deciden cuándo, se explican en otra sección).
- `fsync()` fuerza que todos los datos y metadatos modificados y cacheados de un fichero se escriban en el dispositivo subyacente.
- La llamada se bloquea hasta que el dispositivo confirma que la transferencia se ha completado.
- Llamar a `fsync()` con demasiada frecuencia perjudica el rendimiento (en la práctica fuerza un comportamiento write-through); no llamarla nunca implica el riesgo de perder datos que aún no están en disco si el sistema falla.

Consultar los atributos de un fichero

stat() y fstat()

- `int stat(char *name, struct stat *buf);`
- `int fstat(int fd, struct stat *buf);`
- Argumentos:
  - `name`: nombre del fichero
  - `fd`: descriptor de fichero
  - `buf`: puntero a un objeto de tipo struct stat donde se almacenará el valor de los atributos del fichero
    - `man 2 stat` para consultar campos de struct stat
- Valor de retorno:
  - Cero ó -1 si error

Ejemplo: copia de un fichero (1/2)

file_copy.cpp

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>

#define BUFSIZE 512

void main(int argc, char **argv) {
    int fd_in, fd_out, n_read;
    char buffer[BUFSIZE];

    fd_in = open(argv[1], O_RDONLY); /* open the input file */
    if (fd_in < 0){
        perror("open"); exit(1);
    }

    fd_out = creat(argv[2], 0644); /* create the output file */
    if (fd_out < 0){
        close(fd_in);
        perror("creat"); exit(1);
     }
```

Modo uso

- `copy_file <input-file> <output-file>`

Ejemplo: copia de un fichero (2/2)

file_copy.cpp

```
/* main loop to transfer data between files */
   while ((n_read = read(fd_in, buffer, BUFSIZE)) > 0) {
      /* Transfer data from the buffer onto the output file */
      if (write(fd_out, buffer, n_read) < n_read) {
         perror("write");
         close(fd_in); close(fd_out);
         exit(1);
      }
   }
   if (n_read < 0) {
      perror("read");
      close(fd_in); close(fd_out);
      exit(1);
   }
   close(fd_in); close(fd_out);
   exit(0);
}
```



Servicios POSIX para directorios



- Visión lógica en UNIX: tabla de pares (nombre lógico, num. nodo-i)
- Gestión complicada:
  - Nombres de fichero tienen longitud variable
  - En UNIX, varios sistemas de ficheros "conviven" en un único árbol/grafo de directorios
    - Distintas representaciones de directorios
- Servicios POSIX: operaciones para crear, modificar y recorrer directorios

Operaciones básicas sobre directorios



- **mkdir**: crea un directorio con un nombre y protección
- **rmdir**: borra el directorio vacío con un nombre
- **opendir**: abre un directorio como una secuencia de entradas y se sitúa en la primera
- **readdir**: lee la siguiente entrada del directorio
- **rewinddir**: sitúa el puntero de posición en la primera entrada
- **closedir**: cierra un directorio abierto con un descriptor
- **link/symlink**: crea una nueva entrada en el directorio para un enlace físico o simbólico
- **unlink**: elimina una entrada del directorio
- **rename**: cambiar el nombre de una entrada del directorio
- **chdir**: cambia el directorio de trabajo del proceso
- **getcwd**: obtener el directorio de trabajo

Crear un directorio

mkdir()

- `int mkdir(const char *name, mode*t mode);`
- Argumentos:
  - `name`: nombre del directorio
  - `mode`: bits de protección
- Valor de retorno:
  - Cero ó -1 si error
- Descripción:
  - Crea un directorio de nombre name
  - `UID_propietario` = `UID_efectivo`
  - `GID_propietario` = `GID_efectivo`

Borrar un directorio

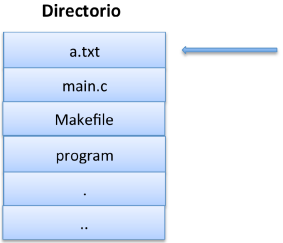
rmdir()

- `int rmdir(const char *name);`
- Argumentos:
  - `name`: nombre del directorio
- Valor de retorno:
  - Cero ó -1 si error
- Descripción:
  - Borra el directorio si está vacío
  - Si el directorio no está vacío, `rmdir()` devuelve -1 (error)

Servicios POSIX para directorios (II)

El directorio se expone al usuario como una tabla (array de struct dirent) con un puntero de posición asociado

```
struct dirent {
        /* file name */
        char *d_name;
        ...
}
```



- Al abrir el directorio (`opendir()`) el puntero apunta a la primera entrada
- `readdir()` devuelve entrada actual y avanza puntero
  - Al llegar a la última entrada, `readdir()` devuelve NULL
- `rewinddir()` permite reubicar el puntero en la primera entrada

Abrir un directorio

`opendir()`

- `DIR *opendir(char *dirname);`
- Argumentos:
  - `dirname`: puntero al nombre del directorio
- Valor de retorno:
  - Un puntero para utilizarse en `readdir()`, `rewindir()` o `closedir()`. NULL si hubo error
- Descripción:
  - Abre un directorio como una secuencia de entradas. Se coloca en el primer elemento

Leer entradas de un directorio

`readdir()`

- `struct dirent *readdir(DIR *dirp);`
- Argumentos:
  - `dirp`: puntero retornado por `opendir()`
- Valor de retorno:
  - Un puntero a un objeto del tipo `struct dirent` que representa una entrada de directorio o NULL si hubo error
- Descripción:
  - Devuelve la siguiente entrada del directorio asociado a `dirp` y avanza el puntero a la siguiente entrada
  - La representación de la entrada de directorio (`struct dirent`) es dependiente de la implementación. Debería asumirse que tan solo se obtiene un miembro: `char *d_name`

Resetear el puntero de posición

`rewinddir()`

- `void rewinddir(DIR *dirp);`
- Argumentos:
  - `dirp`: puntero retornado por `opendir()`
- Descripción:
  - Sitúa el puntero de posición dentro del directorio en la primera entrada

Cerrar directorio

`closedir()`

- `void closedir(DIR *dirp);`
- Argumentos:
  - `dirp`: puntero retornado por `opendir()`
- Valor de retorno:
  - Cero ó -1 si error
- Descripción:
  - Destruye la asociación entre `dirp` y la secuencia de entradas de directorio

Crear una entrada de directorio

`link()` y `symlink()`

- `int link(const char *existing, const char *new);`
- `int symlink(const char *existing, const char *new);`
- Argumentos:
  - `existing`: nombre del fichero existente
  - `new`: nombre de la nueva entrada de directorio
- Valor de retorno:
  - Cero ó -1 si error
- Descripción:
  - Crea un nuevo enlace, físico o simbólico, para un fichero existente
  - El sistema no registra cuál es el enlace original
  - `existing` no debe ser el nombre de un directorio

Leer el destino de un enlace simbólico

`readlink()`

- `int readlink(const char *path, char *buf, size*t bufsize);`
- Lee la ruta almacenada dentro de un enlace simbólico, guardándola en `buf`
- La cadena devuelta en `buf` **no** termina en carácter nulo
- El tamaño necesario para `buf` se puede determinar con `lstat(2)` (el campo `st_size` del fichero, de la diapositiva de la estructura `stat`)
- Esta es la llamada al sistema que hay detrás del comando `readlink(1)`. Es también lo que el SO invoca internamente, en nombre del usuario, cada vez que la resolución de una ruta atraviesa un enlace simbólico.

Eliminar una entrada de directorio

`unlink()`

- `int unlink(const char* path);`
- Argumentos:
  - `path`: nombre del archivo
- Valor de retorno:
  - Devuelve 0 ó -1 si error
- Descripción:
  - Elimina la entrada de directorio y decrementa el número de enlaces del archivo correspondiente
  - Cuando el número de enlaces es igual a cero y ningún proceso lo mantiene abierto, se libera el espacio ocupado por el fichero y el fichero deja de ser accesible

Renombrar una entrada de directorio

`rename()`

- `int rename(char *old, char *new);`
- Argumentos:
  - `old`: nombre de un fichero existente
  - `new`: nuevo nombre del fichero
- Valor de retorno:
  - Cero ó -1 si error
- Descripción:
  - Cambia el nombre del fichero `old`. El nuevo nombre es `new`

Cambiar el directorio de trabajo

chdir()

- `int chdir(char *name);`
- Argumentos:
  - name‘: nombre de un directorio
- Valor de retorno:
  - Cero ó -1 si error
- Descripción:
  - Modifica el directorio de trabajo actual, aquel a partir del cual se forman los nombre relativos

Obtener ruta del directorio de trabajo

`getcwd()`

- `char *getcwd(char *buf, size_t size);`
- Argumentos:
  - `buf`: puntero al espacio donde almacenar el nombre del directorio actual
  - `size`: longitud en bytes de dicho espacio
- Valor de retorno:
  - Puntero a `buf` o `NULL` si error
- Descripción:
  - Obtiene el nombre del directorio actual

Programa que lista un directorio (1/2)

file_copy.cpp

```
#include <sys/types.h>
#include <dirent.h>
#include <stdio.h>

#define MAX_BUF 256

void main(int argc, char **argv){
  DIR *dirp;
  struct dirent *dp;
  char buf[MAX_BUF];

  /* Prints the path of the current working directory */
  getcwd(buf, MAX_BUF);

  printf("Current working directory: %s\n", buf);
  ...
```

Modo uso

- `list_dir <directory>`

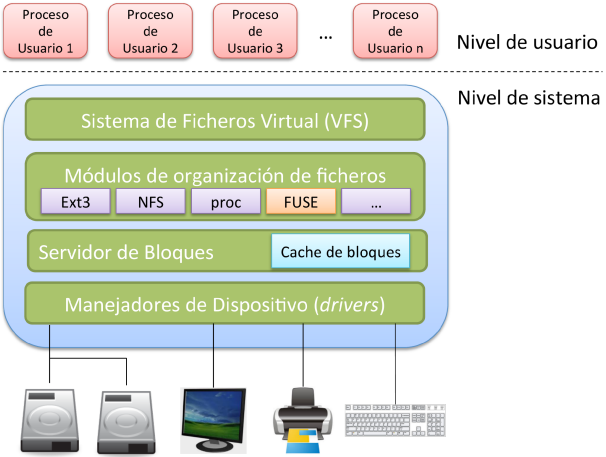
Programa que lista un directorio (2/2)

file_copy.cpp

```
    /* Open the directory passed as the first argument */
    dirp = opendir(argv[1]);

    if (dirp == NULL) {
        fprintf(stderr,"Couldn't open %s\n", argv[1]);
    } else {
        /* Iterate the set of directory entries */
        while ( (dp = readdir(dirp)) != NULL)
            printf("%s\n", dp->d_name);
      closedir(dirp);
   }
   exit(0);
}
```

Arquitectura del Sist. de Gestión de Ficheros



Componentes del Sist. de Gestión de Ficheros

Sistema de ficheros virtual (VFS)

- Capa encargada de proporcionar la interfaz de llamadas al sistemas
  - Manejo de directorios e interpretación de rutas
  - Servicios genéricos de ficheros (`API SO`)
  - Abstracción de los distintos sistemas de archivos. Ficheros abiertos representados por mediante un envoltorio (`wrapper`) del descriptor físico en cuestión: nodo-v (nodo virtual)

Objetos del VFS (I): la cache de dentries

- La resolución de rutas es costosa: cada componente de la ruta cuesta al menos una lectura de disco de los datos de un directorio. El VFS cachea el *resultado* de ese trabajo en un objeto **dentry** (entrada de directorio)
- Un dentry enlaza un componente de la ruta (p. ej., `mike` en `/home/mike`) con el inodo al que resuelve, además de un puntero a su dentry padre, de forma que una ruta completa es una cadena de dentries.
- El conjunto de dentries cacheados es la **cache de dentries (dcache)**: una vez que se ha recorrido una ruta, volver a recorrerla se puede resolver casi por completo desde memoria, sin necesidad de acceder a disco.
- Esta es exactamente la misma técnica que hace que el botón "atrás" de un explorador de ficheros o el autocompletado de rutas en la shell parezcan instantáneos

Objetos del VFS (II): la cache de inodos

- Todo inodo que el VFS ha tocado recientemente se mantiene en memoria en la **cache de inodos**, como un objeto de inodo genérico del VFS que envuelve al específico de cada sistema de ficheros (`ext4`, `NFS`, ...)
- Esto es lo que permite al SO responder a llamadas stat()', comprobaciones de permisos y consultas de tamaño sin tener que volver a disco cada vez
- Los inodos en memoria modificados (dirty) son lo que las políticas de write-back/escritura retardada (más adelante en esta sección) acaban volcando a disco, igual que los bloques de datos de la buffer cache

Objetos del VFS (III): el superbloque

- El objeto **superbloque** contiene información global del sistema de ficheros: tamaño de bloque, número total/libre de bloques e inodos, y punteros a las operaciones específicas de ese sistema de ficheros (cómo implementa *ese* sistema de ficheros `read`, `write`, `allocate`…)
- Es el ancla del VFS para todo lo demás: el dentry raíz, las entradas de la cache de inodos y los bloques de la buffer cache de un montaje dado cuelgan todos de su superbloque
- Este es el equivalente en memoria, a nivel de VFS, del superbloque en disco ya visto en los diagramas de organización en disco (UNIX: Boot / Superbloque / Mapas de bits / Inodos / Ficheros y Directorios), la misma información, ahora mantenida en memoria para un acceso rápido mientras el sistema de ficheros está montado

Componentes del Sist. de Gestión de Ficheros

Módulos de organización de ficheros:

- Implementan los servicios para cada tipo de sistema de ficheros soportado por el SO
  - Un módulo por cada tipo de sistemas de fichero soportado (`UNIX`, `EXT3`, `NTFS`, `FAT` etc.) o pseudo-ficheros como `/proc`
  - Proporcionan algoritmos para traducir direcciones lógicas de bloques a sus correspondientes direcciones físicas
  - Gestionan el espacio de los sistemas de ficheros, la asignación de bloques a ficheros y el manejo de los descriptores internos de fichero (i-nodos de UNIX o entradas de directorio FAT)

Componentes del Sist. de Gestión de Ficheros

Servidor de bloques:

- Emite los mandatos genéricos para leer y escribir bloques a los manejadores de dispositivo
  - Ej: Leer bloque 320 del sistema de ficheros #1→ read*block 320 /dev/sda2
- Implementa mecanismos de optimización de la E/S como la cache de bloques

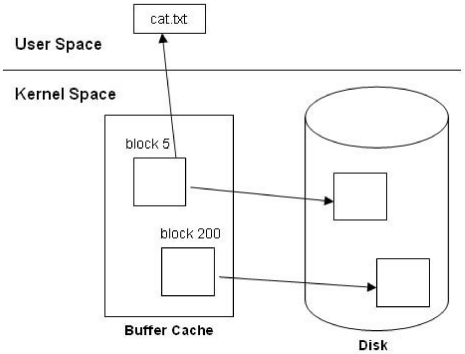
Componentes del Sist. de Gestión de Ficheros

Manejadores de dispositivo (Drivers)

- Recibe ordenes de E/S de alto nivel y las traduce al formato que entiende el controlador HW del dispositivo en cuestión
- Existe un *driver* por cada clase de dispositivo específico que hay en el sistema
  - Para cada dispositivo de almacenamiento hay una cola de peticiones que el *driver* debe planificar adecuadamente para obtener buen rendimiento de la E/S
  - Explotar la localidad de los accesos garantiza un mayor ancho de banda

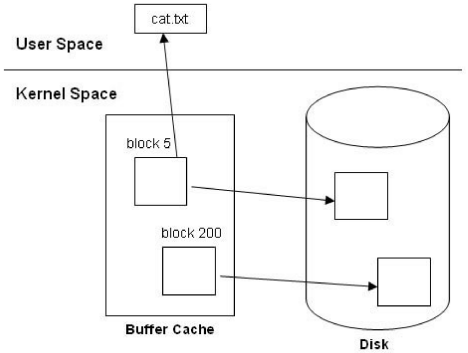
Cache de bloques

- El acceso a disco es órdenes de magnitud más lento que el acceso a RAM
- **Cache de bloques**: Región de memoria que se usa como cache controlada por software para almacenar bloques de los dispositivos de almacenamiento
  - Explota principios de localidad espacial y localidad temporal



Cache de bloques

- Principal problema: consistencia de la información
  - Los datos pueden estar actualizados en la cache de bloques pero no en disco
- El SO debe implementar una **política de reemplazo** y una **política de escritura** para la cache de bloques



Políticas de reemplazo

Escenario

- Cuando el servidor de bloques (SB) recibe una petición de L/E de un bloque específico, debe buscarlo en la cache de bloques
  - En caso de que no esté, se lee del dispositivo y se copia a la cache
  - Si la cache está llena, es necesario hacer hueco para el nuevo bloque reemplazando uno de los existentes

Políticas de reemplazo

- **FIFO** (First In First Out)
- **MRU** (Most Recently Used)
- **LRU** (Least Recently Used)
  - Los bloques más usados se mantienen en la cache (RAM)
  - Política más común

Políticas de escritura (I)

- **Escritura inmediata** (*write-through*): se escribe cada vez que se modifica el bloque
  - No hay problema de fiabilidad, pero se reduce el rendimiento del sistema
- **Escritura diferida** (*write-back*): sólo se escriben los datos a disco cuando se eligen para su reemplazo por falta de espacio en la cache
  - Optimiza el rendimiento, pero genera los problemas de fiabilidad anteriormente descritos

Políticas de escritura (II)

- **Escritura retrasada** (*delayed-write*), que consiste en escribir a disco los bloques de datos modificados en la cache de forma periódica cada cierto tiempo (30 segundos en UNIX)
  - Compromiso entre rendimiento y fiabilidad
  - Reduce la extensión de los posibles daños por pérdida de datos
  - Los bloques especiales se escriben inmediatamente al disco
  - No se puede quitar un disco del sistema sin antes volcar los datos de la cache
- **Escritura al cierre** (*write-on-close*): cuando se cierra un fichero, se vuelcan al disco los bloques del mismo que tienen datos actualizados